

Sentence Pair Classification

Ioana-Livia Popescu
Artificial Intelligence - Group 407
University of Bucharest
Romania
ioana.popescu1@s.unibuc.ro

Abstract

This documentation presents my approach regarding a sentence pair classification challenge focused on training classifiers using a dataset composed of Romanian sentences.

1 Introduction

1.1 Task description

Sentence pair classification consists in classifying pairs of sentences based on a given set of labels. The task at hand consisted in classifying pairs of sentences based on four possible labels (0, 1, 2, 3). The presented approaches are based on Practical Machine Learning Lectures (University of Bucharest, 2023) and official sklearn (skl) documentation .

1.2 Evaluation metrics

The evaluation metric on the test set is the macro F1 score, where

$$F1 = \frac{2 \times P \times R}{P + R}$$

where P is the precision and R is the recall.

During experiments, models will trained on the train subset and evaluated on the validation subset in terms of precision, recall and macro F1 score. The final submissions will consist in models that performed best trained on a different subset obtained by concatenating the train and validation subsets.

2 Data Analysis

2.1 Initial data distribution

subset	0	1	2	3	size
Train	2592	1300	25722	28500	58114
Validation	74	72	1135	1778	3059
Test	-	-	-	-	3000

Table 1: Initial data distribution

2.1.1 Observations

The dataset is imbalanced in terms of the distribution of labels.

subset	0	1	2	3
Train	< 5%	< 3%	> 40%	> 40%
Validation	< 3%	< 3%	> 30%	> 50%

Table 2: Initial data distribution - procentual approximation

This aspect is important given that the evaluation metric is macro F1 which is computed by taking the arithmetic mean of all the per-class F1 scores. As a result, the classes equally impact the result - low F1 score for any of the classes will have a significant effect.

2.2 Null values

Upon verification, there are no null values in the dataset.

2.3 Sentence length

2.3.1 Characters

length	sentence1	sentence2
Minimum	1	2
Maximum	13076	5355
Average	158.94	153.54

Table 3: Character based sentence length analysis - train subset

2.3.2 Words

length	sentence1	sentence2
Minimum	1	1
Maximum	1507	595
Average	24.52	23.78

Table 4: Word based sentence length analysis - train subset

2.3.3 Observations

The word based sentence length analysis is based on splitting the sentences by ' ' - sentence lengths

are of decent size.

3 Data preprocessing

3.1 Approach

In terms of data preprocessing, the `TfidfVectorizer` - presented in below section - has the parameter `lowercase` set to its default value (`True`); as a result, it converts all characters to lowercase before tokenizing. Other preprocessing techniques that have been tested are: removing punctuation, removing possible hyperlinks, emojis and extra white spaces.

3.2 Results

None of the preprocessing methods mentioned above resulted in a significant improvement of the results. The final submissions are based on data that has only been converted to lowercase before tokenization - performed with `Tfidf`.

4 Feature representation

The utilized features have been obtained with the help of `TfidfVectorizer` (`tfi`) which converts raw text into a matrix of tf-idf features - in the current context, for each example, values of `'sentence1'` and `'sentence2'` are concatenated before the vectorizer is applied. I chose `TfidfVectorizer` because, unlike `CountVectorizer` which only takes into account how many times a word appears in the document, `TfidfVectorizer` also considers how important that word is - it aims to scale down the impact of tokens that occur very frequently and aren't as informative as features that rarely occur in the data. In order to better capture the context, experiments with various values for `ngram_range` have been conducted (ranging from (1,2) to (1,7)) - which determines the length range of n-grams (sequences of n adjacent symbols in particular order). The best results have been obtained for values around (1,5) with the default analyzer (`'word'`).

5 Tested models

model	public score	private score
Multinomial Naive Bayes	0.67688	0.65761
Random Forest Classifier	0.67688	0.65118
Complement Naive Bayes	0.67602	0.62579
SVM	0.66377	0.63982

Table 5: Best scores based on competition submissions

For the final standings, I selected the Multinomial Naive Bayes and Random Forest Classifier

approaches with the highest public scores. The Multinomial Naive Bayes approach ranked the 1st Place on the final standings and the Random Forest would have ranked in top 30.

5.1 Multinomial Naive Bayes

5.1.1 Approach

The classification task considers four possible classes, therefore suitable for a Multinomial Naive Bayes approach which implements the Naive Bayes algorithm for data following a multinomial distribution - also known to perform well with tf-idf features in practice. I used the implementation made available by `sklearn` (`mNB`). In order to find the optimal value for `alpha`, I performed multiple hyperparameter tuning experiments through `GridSearch` (`gri`) and found the `alpha` values of 0.1 and 0.15 to be associated with best estimators - alongside features of word n-gram range of (1,5). The values for the parameters - except `alpha` - are the default ones as experiments conducted with alternative values did not provide better results.

5.1.2 Experiments

index	n_grams	alpha	force_alpha	fit_prior	class_prior
1	(1,5)	0.1	'warn'	True	None
2	(1,5)	0.15	'warn'	True	None
3	(1,7)	0.1	'warn'	True	None
4	(1,4)	0.1	'warn'	True	None
5	(1,3)	0.1	'warn'	True	None
6	(1,2)	0.1	'warn'	True	None

Table 6: Experiments with Multinomial Naive Bayes

index	F1_macro (train)	F1_macro (val)
1	0.99	0.37
2	0.93	0.36
3	1.00	0.37
4	0.98	0.37
5	0.92	0.36
6	0.73	0.35

index	F1_macro(train+ val)
1	0.99
2	0.93
3	1.00
4	0.98
5	0.92
6	0.72

Table 7: Experiments with Multinomial Naive Bayes - F1 macro scores

index	public score	private score
1	0.67688	0.65761
2	-	-
3	0.67688	0.656
4	-	-
5	-	-
6	-	-

Table 8: Competition submission scores

Accuracy	1.00	-	-
Macro Avg	1.00	0.99	0.99
Weighted Avg	1.00	1.00	1.00
Support	-	-	61173

5.1.3 Best (selected) model - Classification reports

Table 9: Classification Report - train

Class	Metrics			Support
	Precision	Recall	F1-Score	
0	1.00	0.99	0.99	2592
1	1.00	0.96	0.98	1300
2	1.00	1.00	1.00	25722
3	1.00	1.00	1.00	28500

Accuracy	1.00	-	-
Macro Avg	1.00	0.99	0.99
Weighted Avg	1.00	1.00	1.00
Support	-	-	58114

Table 10: Classification Report - validation

Class	Metrics			Support
	Precision	Recall	F1-Score	
0	0.25	0.01	0.03	74
1	1.00	0.04	0.08	72
2	0.55	0.73	0.63	1135
3	0.78	0.68	0.73	1778

Accuracy	0.67	-	-
Macro Avg	0.65	0.37	0.37
Weighted Avg	0.69	0.67	0.66
Support	-	-	3059

Table 11: Classification Report - train + val

Class	Metrics			Support
	Precision	Recall	F1-Score	
0	1.00	0.99	0.99	2666
1	1.00	0.96	0.98	1372
2	1.00	1.00	1.00	26857
3	1.00	1.00	1.00	30278

5.1.4 Best (selected) model - Confusion Matrices

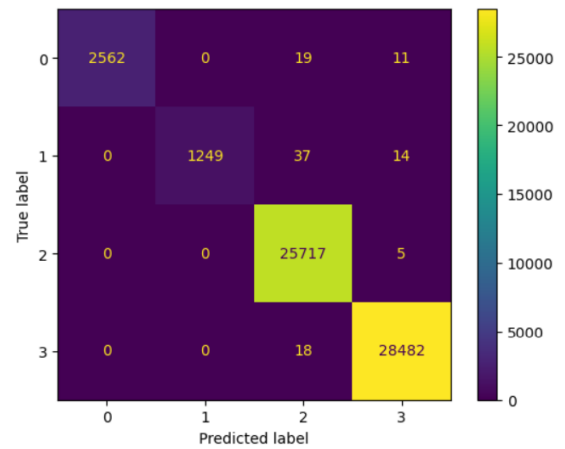


Figure 1: Confusion Matrix for evaluation on train - trained on train

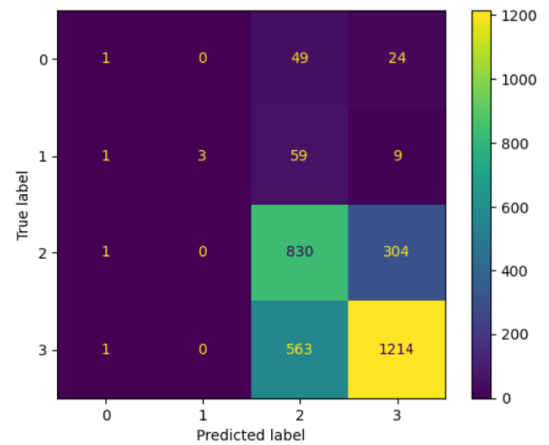


Figure 2: Confusion Matrix for evaluation on validation - trained on train

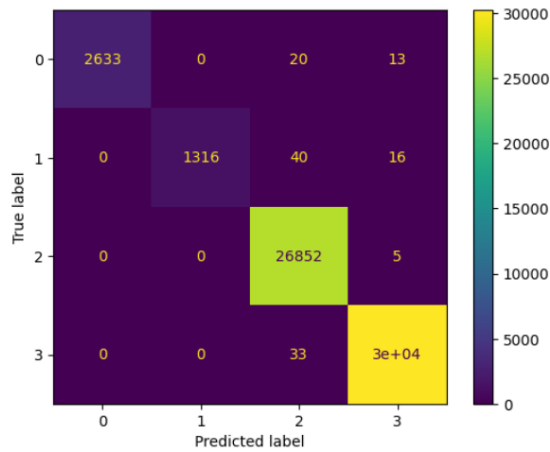


Figure 3: Confusion Matrix for evaluation on train + val - trained on train + val

5.2 Complement Naive Bayes

5.2.1 Approach

After experimenting with Multinomial Naive Bayes, the next step represented moving onto Complement Naive Bayes (Com) - an adaptation of the standard Multinomial Naive Bayes algorithm which computes the model's weights based on statistics from the complement of each class. Due to this property, it generally performs well on imbalanced data sets - which is the case for the dataset at hand. Upon conducting experiments, I haven't been able to obtain better results than the previously obtained scores - using the Multinomial Naive Bayes approach.

5.2.2 Experiments

index	n_grams	alpha	force_alpha	fit_prior
1	(1,5)	1.0	'warn'	True
2	(1,7)	1.0	'warn'	True
3	(1,7)	1.0	'warn'	True

index	class_prior	norm
1	None	False
2	None	True
3	None	False

Table 12: Experiments with Complement Naive Bayes

index	F1_macro (train)	F1_macro (val)
1	0.94	0.37
2	0.99	0.37
3	0.99	0.37

index	F1_macro(train+ val)
1	0.94
2	0.99
3	0.99

Table 13: Experiments with Complement Naive Bayes - F1 macro scores

index	public score	private score
1	0.67602	0.62579
2	-	-
3	-	-

Table 14: Competition submission scores

5.3 Random Forest Classifier

5.3.1 Approach

A Random Forest Classifier is an ensemble machine learning model that combines multiple decision trees to make accurate predictions - each tree is trained on a random subset of the data. The final prediction is determined by the majority vote of the individual trees (the used implementation is the one offered by sklearn (rf)). The n_jobs parameter has been set to -1 in order to use all processors. The values used for criterion during the experiments are 'gini' and 'entropy' which led to similar results.

5.3.2 Experiments

index	n_grams	criterion	class_weight	n_jobs
1	(1,7)	'gini'	'balanced'	-1
2	(1,5)	'gini'	'balanced'	-1
3	(1,5)	'gini'	None	-1
4	(1,5)	'entropy'	'balanced'	-1

Table 15: Experiments with Random Forest Classifier

index	F1_macro (train)	F1_macro (val)
1	1.00	0.32
2	1.00	0.33
3	1.00	0.34
4	1.00	0.33

index	F1_macro(train+ val)
1	1.00
2	1.00
3	-
4	-

Table 16: Experiments with Random Forest Classifier - F1 macro scores

index	public score	private score
1	0.67688	0.65118
2	-	-
3	-	-
4	-	-

Table 17: Competition submission scores

Accuracy	1.00	-	-
Macro Avg	1.00	1.00	1.00
Weighted Avg	1.00	1.00	1.00
Support	-	-	61173

5.3.3 Best (selected) model - Classification reports

Table 18: Classification Report - train

Class	Metrics			Support
	Precision	Recall	F1-Score	
0	1.00	1.00	1.00	2592
1	1.00	1.00	1.00	1300
2	1.00	1.00	1.00	25722
3	1.00	1.00	1.00	28500

Accuracy	1.00	-	-
Macro Avg	1.00	1.00	1.00
Weighted Avg	1.00	1.00	1.00
Support	-	-	58114

Table 19: Classification Report - validation

Class	Metrics			Support
	Precision	Recall	F1-Score	
0	0.50	0.01	0.03	74
1	1.00	0.03	0.05	72
2	0.48	0.92	0.63	1135
3	0.89	0.43	0.58	1778

Accuracy	0.59	-	-
Macro Avg	0.72	0.35	0.32
Weighted Avg	0.73	0.59	0.57
Support	-	-	3059

Table 20: Classification Report - train + val

Class	Metrics			Support
	Precision	Recall	F1-Score	
0	1.00	1.00	1.00	2666
1	1.00	1.00	1.00	1372
2	1.00	1.00	1.00	26857
3	1.00	1.00	1.00	30278

5.3.4 Best (selected) model - Confusion Matrices

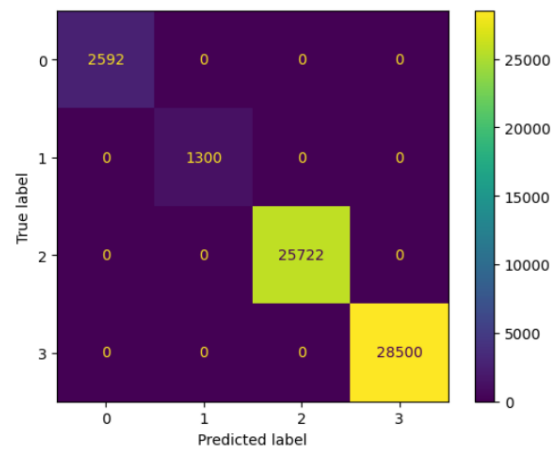


Figure 4: Confusion Matrix for evaluation on train - trained on train

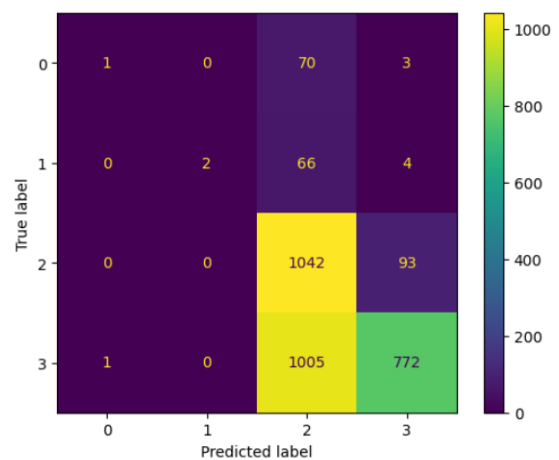


Figure 5: Confusion Matrix for evaluation on validation - trained on train

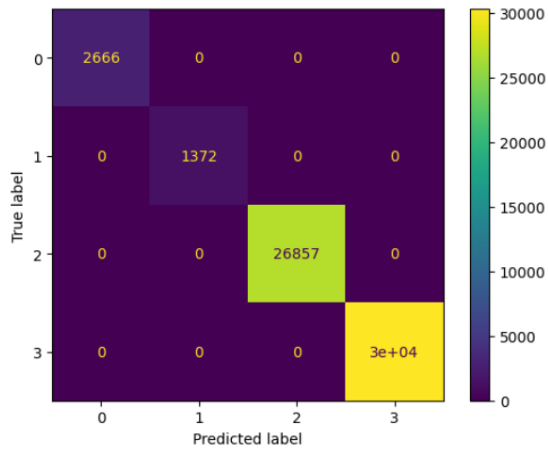


Figure 6: Confusion Matrix for evaluation on train + val
- trained on train + val

5.4 SVM

5.4.1 Approach

In regards to the SVM approach, I utilized the Support Vector Classification implementation offered by sklearn (`svc`) which is appropriate for classification tasks - it finds the optimal hyperplane that separates the classes in the feature space. Due to the imbalanced nature of the dataset, I assumed the model - capable of handling complex decision boundaries - will perform well. During the experiments, models that used the default 'rbf' kernel outperformed the ones that used the 'linear' kernel, signaling that the linear kernel was not able to capture the inherent non-linear relationships present in the data.

5.4.2 Experiments

index	n_grams	C	kernel	class_weight	max_iter
1	(1,5)	1	'rbf'	None	10000
2	(1,5)	10	'rbf'	None	10000
3	(1,5)	3	'rbf'	None	10000
4	(1,5)	2	'rbf'	None	10000
5	(1,5)	3	'linear'	None	10000
6	(1,5)	5	'linear'	None	10000
7	(1,5)	3	'linear'	'balanced'	10000

index	trained_on
1	train
2	train
3	train+val
4	train+val
5	train+val
6	train+val
7	train+val

Table 21: Experiments with Support Vector Classification

index	F1_macro (train)	F1_macro (val)	F1_macro(train+ val)
1	0.86	0.34	-
2	0.98	0.35	-
3	0.98	0.36	0.98
4	-	-	0.98
5	0.98	0.40	-
6	0.98	0.40	-
7	0.98	0.36	0.98

Table 22: Experiments with Support Vector Classification - F1 macro scores

index	public score	private score
1	0.61009	0.59226
2	0.66342	0.62537
3	0.65951	0.63772
4	0.67064	0.63572
5	0.65691	0.63424
6	0.66377	0.63982
7	0.65959	0.63744

Table 23: Competition submission scores

References

Complementnb. Last Accessed 29.12.2023 12:00 PM EEST https://scikit-learn.org/stable/modules/generated/sklearn.naive_bayes.ComplementNB.html.

Gridsearch (sklearn). Last Accessed 29.12.2023 12:00 PM EEST https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.GridSearchCV.html.

Multinomialnb (sklearn). Last Accessed 29.12.2023 12:00 PM EEST https://scikit-learn.org/stable/modules/generated/sklearn.naive_bayes.MultinomialNB.html.

Randomforestclassifier (sklearn). Last Accessed 29.12.2023 12:00 PM EEST <https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestClassifier.html>.

sklearn. Last Accessed 29.12.2023 12:00 PM EEST <https://scikit-learn.org/stable/>.

Svc. Last Accessed 29.12.2023 12:00 PM EEST <https://scikit-learn.org/stable/modules/generated/sklearn.svm.SVC.html>.

Tfidfvectorizer (sklearn). Last Accessed 29.12.2023 12:00 PM EEST https://scikit-learn.org/stable/modules/generated/sklearn.feature_extraction.text.TfidfVectorizer.html.